
Übungsblatt 8

Abgabe: 15.06.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Allgemeine Hinweise zur Implementierung von Fenstern:

- Fenster sind ausschließlich in AWT (Layout und Ereignisbehandlung) und Swing (grafische Komponenten) zu programmieren.
- Es sind immer genau die benötigten Importe anzugeben (keine Importe mit dem *-Operator).
- Auch wenn nicht explizit gefordert, sind Fenster immer als eigene Unterklassen einer API-Fensterklasse zu implementieren (siehe Vorlesungsskript und -beispiele).

Aufgabe 29 ** (*Knoten mit optionalem Elternknoten, 35 Minuten*)

In dieser Aufgabe sollen Sie eine Datenklasse **Node** für Knoten in einer Baumstruktur implementieren. Ein Knoten besitzt einen Bezeichner (eine nicht-leere Zeichenkette) als eindeutigen Identifikator: Zwei Knoten sind genau dann gleich, wenn ihr Bezeichner gleich ist. Außerdem kann ein Knoten einen Elternknoten besitzen, muss aber nicht: Der Wurzelknoten eines Baumes hat beispielsweise keinen Elternknoten. Es sollen Methoden zur Verfügung gestellt werden, um diese Informationen auszulesen und zu bearbeiten. `set-` und `link-`Methoden sollen bei ungültigen Werten die ungeprüfte API-Ausnahme `IllegalArgumentException` werfen. Damit nach außen hin nicht der Wert `null` verwendet werden muss, soll der Elternknoten in einem `Optional`-Objekt gekapselt zurückgegeben werden. Eine Einschränkung an die Beziehung zu einem Elternknoten ist, dass Bäume keine Kreise enthalten dürfen: Was das genau bedeutet, wird in Teilaufgabe c) erklärt.

a) (*, *Klassenkarte entwerfen, 8 Minuten*)

Entwerfen Sie eine Klassenkarte für Knoten. Achten Sie darauf, Elternknoten nicht als Attribut, sondern als Rolle in einer Objektbeziehung zu modellieren.

b) (**, *Klasse implementieren, 14 Minuten*)

Implementieren Sie die Klasse **Node** bis auf die Methode `checkParent`. Vergessen Sie nicht den Konstruktor sowie passende Datentypen. Mit dem Konstruktor kann nur der Bezeichner gesetzt werden – der Elternknoten hat nach dem Erstellen des Objekts immer den Wert `null`. Achten Sie darauf, dass die Methode `getParent` ein `Optional<Node>`-Objekt zurückliefert, das den Elternknoten kapselt. Ist kein Elternknoten gesetzt, soll ein leeres `Optional`-Objekt zurückgegeben werden.

c) (**, *checkParent* implementieren, 6 Minuten)

Implementieren Sie die `checkParent`-Methode, in der die folgenden Überprüfungen stattfinden, wenn *P* als Elternknoten von *C* gesetzt werden soll:

- *P* darf nicht `null` sein (der Elternknoten kann mit der `unlink`-Methode entfernt werden).
- *C* kann nicht sein eigener Elternknoten sein.
- *C* darf nicht bereits der Elternknoten von *P* sein.
- Wenn *P* bereits einen Elternknoten *P'* hat, darf *C* auch nicht dessen Elternknoten sein usw.
- Diese letzte Überprüfung muss für die gesamte Kette an Elternknoten über *P* durchgeführt werden.

Hinweis: Da für diese `check`-Methode Eigenschaften des konkreten Objekts relevant sind, kann diese Methode keine Klassenmethode sein!

d) (*, *Programmklasse implementieren*, 7 Minuten)

Implementieren Sie eine Programmklasse `Tree`, mit der Sie Ihre `Node`-Klasse testen können:

- Erzeugen Sie drei Knoten `"root"`, `"child"` und `"grandchild"`.
- Legen Sie die Elternbeziehungen so fest, dass `"grandchild"` den Knoten `"child"` und `"child"` den Knoten `"root"` als Elternknoten besitzt.
- Geben Sie für alle drei Knoten auf der Kommandozeile aus, ob sie einen Elternknoten besitzen. Falls ja, geben Sie zusätzlich den Bezeichner des Elternknotens aus.
- Versuchen Sie anschließend, `"grandchild"` als Elternknoten von `"root"` zu setzen, und fangen Sie die geworfene `IllegalArgumentException` ab. Geben Sie eine entsprechende Meldung auf der Kommandozeile aus, um zu zeigen, dass Zyklen verhindert werden.
- Entkoppeln Sie schließlich `"child"` von seinem Elternknoten und geben Sie erneut aus, ob `"child"` einen Elternknoten besitzt.

Aufgabe 30 ** (*Fenster ohne Ereignisbehandlung*, 20 Minuten)

In dieser Aufgabe sollen Sie die grundlegenden Elemente von Fenstern in Java näher kennen lernen.

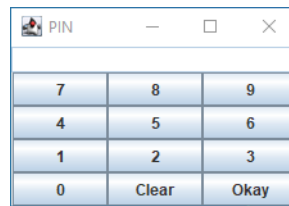
a) (*, Eingabe mit Textfeld, 8 Minuten)

Implementieren Sie ein Hauptanwendungsfenster `AccountNumberFrame` nach folgenden Vorgaben:

- Es soll den Titel `AccountNumberFrame` haben.
- Es soll im Norden von links nach rechts angeordnet die Beschriftung `Accountnumber` und ein aktiviertes einzeliges Textfeld der Breite 20 anzeigen. Beschriftung und Textfeld sollen beide dieselbe Schrift mit Schriftgröße 20 haben.
- Es soll im Süden von links nach rechts angeordnet zwei Buttons mit dem Text `Okay` und `Cancel` anzeigen.
- Der Nord- und der Südbereich sollen unterschiedliche Graufarben als Hintergrundfarbe haben.
- Durch Klicken auf das Fensterkreuz soll die Anwendung beendet werden.
- Das Fenster soll zu Beginn passend groß für die enthaltenen Komponenten sein.
- Implementieren Sie darüber hinausgehend **keine** Ereignisbehandlung.

b) (**, Visuelle Vorgabe, 12 Minuten)

Implementieren Sie ein Hauptanwendungsfenster **PINField**, das möglichst so wie in folgendem Bild aussieht:



Durch Klicken auf das Fensterkreuz soll die Anwendung beendet werden.

Hinweis: Die weiße Fläche im oberen Teil des Fensters soll dazu genutzt werden, die Eingabe anzuzeigen (diese Funktionalität muss *nicht* implementiert werden). Auch für die Buttons soll keine Ereignisbehandlung implementiert werden.

Aufgabe 31 ** (Fenster mit Ereignisbehandlung (*ActionEvents*), 20 Minuten)

a) (*, 10 Minuten)

Implementieren Sie ein Hauptanwendungsfenster **TwoButtons** nach folgenden Vorgaben:

- Es soll den Titel **TwoButtons** haben.
- Es soll im Westen eine Schaltfläche mit dem Text **Left Button** haben.
- Es soll im Osten eine Schaltfläche mit dem Text **Right Button** haben.
- Es soll im Süden ein deaktiviertes mehrzeiliges Textfeld mit 20 Spalten, 5 Zeilen und mit Rollbalken haben.
- Drückt der Benutzer auf eine der Schaltflächen, so soll das zugehörige Kommando in einer neuen Zeile an den bisher im Textfeld angezeigten Text angehängt werden. Implementieren Sie dazu das Hauptanwendungsfenster als Ereignisabhörer.
- Durch Klicken auf das Fensterkreuz soll die Anwendung beendet werden.

b) (**, 5 Minuten)

Implementieren Sie ein Hauptanwendungsfenster **TwoButtonsTwo** nach folgenden Vorgaben:

- Es soll den Titel **TwoButtonsTwo** haben, ansonsten aber genauso wie in Teilaufgabe a) aussehen und auf Benutzeraktionen reagieren.
- Allerdings soll diesmal nicht das Hauptanwendungsfenster als Ereignisabhörer fungieren. Stattdessen soll für jede Schaltfläche ein separater Ereignisabhörer in Form eines Lambda-Ausdrucks implementiert werden.

c) (**, 5 Minuten)

Implementieren Sie ein Hauptanwendungsfenster **TwoButtonsThree** nach folgenden Vorgaben:

- Es soll den Titel **TwoButtonsThree** haben, ansonsten aber genauso wie in Teilaufgabe a) aussehen und auf Benutzeraktionen reagieren.
- Allerdings soll diesmal nicht das Hauptanwendungsfenster als Ereignisabhörer fungieren. Stattdessen soll eine separate Klasse **MyActionListener** als Ereignisabhörer implementiert und benutzt werden.

Hinweis: Übergeben Sie dabei an den Konstruktor von **MyActionListener** eine Referenz auf das Textfeld.

Aufgabe 32 (*Fenster mit Ereignisbehandlung (`WindowEvents`, `KeyEvents`, `JOptionPane`), 30 Minuten*)

In dieser Aufgabe lernen Sie den Umgang mit `WindowEvents`, `KeyEvents` und `JOptionPane` kennen.

a) (*`WindowEvent`, **, 13 Minuten*)

Implementieren Sie ein Hauptanwendungsfenster `DontMinimizeMe` nach folgenden Vorgaben:

- Zu Beginn soll das Fenster den Titel "**Don't minimize me!**" tragen.
- Das Fenster soll die Größe 400×100 Pixel haben.
- Ein Klick auf das Fensterkreuz soll das Fenster schließen.
- Im Zentrum des Fensters soll ein zu Beginn leeres `GridLayout` mit 3 Spalten sein.
- Immer, wenn das Fenster minimiert wird, soll ein neuer Button mit dem Text "**ouch**" in das Raster eingefügt werden.
- Wenn das Fenster aus der Minimierung zurückgeholt wird, soll überprüft werden, wieviele Buttons bereits enthalten sind. Wenn mehr als 3 Komponenten enthalten sind, soll der Titel zu "**Stop iconifying me**" geändert werden, und bei mehr als 6 Komponenten zu "**Dude, seriously, stop**".

Hinweis: Verwenden Sie für die Implementierung der Ereignisbehandlung eine passende anonyme innere Unterklasse von `WindowAdapter`.

b) (*`KeyEvent`, **, 10 Minuten*)

Implementieren Sie ein Hauptanwendungsfenster `KeyCounter` nach folgenden Vorgaben:

- Das Fenster soll zu Beginn den Titel "**KeyCounter: 0**" tragen.
- Immer, wenn nur die Taste '**P**' gedrückt wird, soll die im Titel angezeigte Zahl um 1 erhöht werden.
- Wird '**P**' zusammen mit der Shift-Taste gedrückt, soll die im Titel angezeigte Zahl um 10 erhöht werden.
- Ein Klick auf das Fensterkreuz soll das Fenster schließen.

c) (*`JOptionPane`, *, 7 Minuten*)

Implementieren Sie eine Hauptprogrammklasse `CountDown`, die

- mithilfe eines Eingabe-Dialogs um die Eingabe einer positiven ganzen Zahl bittet und
- danach nacheinander genauso viele Nachrichten-Dialoge öffnet, die von der eingegebenen Zahl an herunterzählen.
- Die Eingabe der Zahl 5 soll also dazu führen, dass nacheinander Dialoge mit den Nachrichten 5, 4, 3, 2 und 1 geöffnet werden, d.h. der nächste Dialog öffnet sich erst, nachdem der vorherige geschlossen wurde.
- Die Eingabe soll so lange wiederholt werden, bis ein gültiger Wert eingegeben wird:
 - Fangen Sie dazu einerseits die auftretende Ausnahme ab, wenn keine Zahl eingegeben wird.
 - Beim Abfangen der Ausnahme soll ein Fehlerdialog mit einer passenden Nachricht ausgegeben werden.
 - Wird andererseits eine nicht-positive Zahl eingegeben, werfen Sie dieselbe Art von Ausnahme, um die Ausnahmebehandlung zu vereinfachen.